



CS
2nd
material

Network



Summary on Lecture 2

--> ايه هي ال Network Application ؟

هي عبارة عن اني بكتب برنامج بيشتغل علي different end systems لازم بيبقي قصاده جهاز علشان يبعثوا ويستقبلوا رسائل من بعض عن طريق بروتوكول معين

- عندنا طريقتين لل Network application :-

1- Client-Server paradigm :

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

ايه معناه؟

اننا بنقسم الابلكيشن لنوعين ..

- ابلكيشنز بيبقي معاها فايلز و databases وبتعمل processing ودي بيبقي اسمها ال Server ولازم

يبقي ليها IP وف حالة ال Server ال IP بيبقي ثابت مش بيتغير وغالبا بتتحتفظ data centers ودي بتبقي اماكن مخصصه لاني احط فيها ال Servers دي

clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other

- ابلكيشنز تانيه بيبقي اسمها Client ودي بتبقي السوفت ويرير ال بيتواصل مع ال Servers وال IP بتاعها بيتغير

يعني ايه do not communicate directly with each other? يعني لنفترض ان هتبعث ميل لواحد

صاحبك ف الحقيقه انت مش بتبعثله مباشر انت بتبعث الميل لل ميل سيرفر بتاعه وبعددين هو ي لوج ان ع الميل سيرفر بتاعه ويستقبل الميل ال انت باعته

2- Peer-Peer architecture :

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability*— new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management

في حاله ال peer-peer مش بيبقي في server السوفت ويريرز بتكلم بعض مباشرة زي ال Voices كدا لو بتكلم حد علي واتساب او ماسنجر ال voices دي مش بتتخزن ف حته بتبقي ف ساعتها كدا

process: program running within a host

clients, servers

client process: process that initiates communication

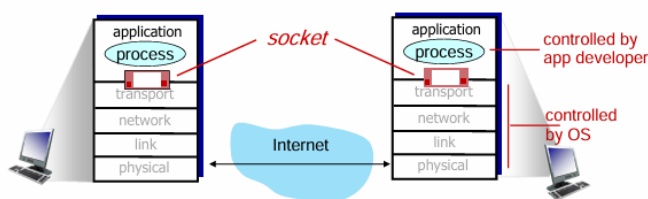
server process: process that waits to be contacted

- ال Process Communicating :

البرامج ال شغاله بيبيعنوا ويستقبلوا داتا ال client-process هي ال بتبتي ال communication ال server-process هي ال مستنيه يجيلها طلبات علشان ترد عليها

→ **Note:** applications with P2P architectures have client processes & server processes

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



❖ طب عمليه ال

Communication دي

بتتم ازاي؟

عن طريق استخدام

library معينه اسمها

Socket كأنها بوابه بين

ال application layer

وال Transport Layer

- to receive messages, process must have **identifier**

طيب الوقتي احنا عارفين ان كل جهاز بيبيقي له Unique Address

ال هو ال IP Address وبيبيقي identifier وبيبيقي عبارة عن binary number مكن من 32-bit

- identifier** includes both **IP address** and **port numbers** associated with process on host.

--> هل ال IP لوحده كفايه اني اوصل المعلومه؟ اكيد لا هو بيديني انا رايع فين بس رايع لمين بقي؟ ايه ضمني ان مفيش اكثر من جهاز شغالين علي نفس ال host؟

علشان كذا ال IP بيبيقي معاه **Port Number** ودا عبارة عن رقم عادي السيرفر بيحجزه

• طب الابلكيشن لما يجي بيعت المسدج هييعتها بشكل ايه؟

ف الحاله دي لازم ال 2 ابلكيشن ال هيتكلموا مع بعض يكونوا متفقين انهم هيمشوا علي قواعد محدده لشكل ومحتوي المسدج ال راحه جايه الكلام دا بيبيقي مكتوب تحت مسمي App-Layer Protocol بيعرفني ايه نوعيات المسدج ال هتروح وهتيجي

- **Message Syntax**: what fields in messages & how fields are delineated
- **Message Semantics** : meaning of information in fields
- **Rules** : for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

البروتوكول دا في منه نوعين :

- **Open Protocols** : يعني بروتوكولز موجوده زي ال HTTP كذا

- **Proprietary Protocols** : زي ال Skype كذا البروتوكولات دي بتبقي جوا الشركات احنا منعرفش عنها حاجه ومش بيبقي ليها document اقدر اوصله

❖ طبعا الكلام ال فات دا بيرن ف ال Application Layer

طيب لما الابلكيشن دا بيعت مسدج هيبعتها لمين؟ لل Transport Layer

طب ال Transport Layer دي هتعملي ايه؟

اول حاجه هتعملي Data Integrity بمعني

اني لما بعت داتا ال داتا دي ال Transport

Layer هتقطعها وتبعثها مش بس كذا دي

هتأكد الداتا ال اتبعثت دي وصلت ولا

موصلتش ووصلت صح ولا لاء

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

- بعض الابلكيشنز بتحتاج Timing معين يعني انا لما ابعت

داتا انا عايزها توصل ف وقت محدد مينفعش تتأخر عنه

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

- الابلكيشنز بتاعتي عايز ابعتها ب Rates قد ايه؟ هنا دور ال throughput

security

- encryption, data integrity,

- اخر حاجه ال Security

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

هنا مثلا عندي اول ابلكيشن File Transfer اولاً مينفعش يبقي فيه data loss throughput عادي يعني اشتغل علي 1 ميغا 50 ميغا مش هنفرق ومش هحتاج انه يتبعث ف وقت معين

نيجي لل real-time audio/ video ممكن داتا تقع ف النص مش مهم بس له throughput معين ال delays ال بين الحاجه وبعضها بتبقي ف حدود ال 100 msec

→ Internet transport protocols services :

1- TCP (Transfer Control Protocol) Services:

- Reliable transport between sending and receiving process
- Flow control: sender won't overwhelm receiver

- يعني مبعثش داتا اكر من ال يقدر يستقبلها

- Congestion Control: throttle sender when network overloaded

- ببتحكم ف حجم الداتا ال خارجه

- Does not provide: timing, minimum throughput guarantee, security
- Connection-Oriented: setup required between client and server processes

- لو اتنين سوفت ويير عايزين يبعثوا لبعض مسدج باستخدام ال TCP لازم السوفت ويير الاولاني يفتح connection مع السوفت ويير الثاني والسوفت ويير الثاني يوافق علي ال connection دا وبعدين يبدأوا يستقبلوا الداتا ويبعثوها لبعض بقي

2- UDP (User Datagram Protocol) Services :

- Unreliable data transfer between sending and receiving process
- Does not provide: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

- اخف بكثير من ال TCP ومش محتاج connection اصلا

→ Securing TCP :

- Vanilla TCP & UDP sockets:

- No encryption
- Cleartext passwords sent into socket traverse Internet in cleartext (!)
- مش بيبقي في تشفير للمعلومات ودا خطر لأن البيانات بتسافر عبر الإنترنت ممكن تكون مكشوفة لأي حد يعني أي شخص عنده إمكانية الوصول للشبكة يقدر يشوف البيانات اللي بتتبع بسهولة لأنها مش مشفرة.

- Transport Layer Security (TLS) :

- Provides encrypted TCP connections
- Data integrity
- End-point authentication
- دا بروتوكول تأمين وتشفير بيعمل تشفير على الداتا وبيضمن ان الداتا ال مبعوته ميثم عليها اي تعديل و بيوفر إمكانية التحقق من هوية الطرف اللي بيتواصل معاك لضمان إنك بتكلم الشخص أو النظام الصحيح.

* TLS implemented in application layer

→ Web and HTTP :

- بحتاج اجيب ويب بيدج عبارة عن صفحه فيها شوية objects ودايما بتحتوي على base HTML File وجواه بيشير الى ال objects المختلفه
- اي اوبجيكت بيبقي متعرف ب URL بيبقي عبارة عن اسم الويبسايت نفسه (Host Name) + ال path name ال path بتاع الفايل ال عايزه من الويب بيدج دي

○ ال HTTP (Hypertext Transfer Protocol) :

- هو بروتوكول بيستخدم في الابلكيشنز ال بنسماها web browsers (Client Server) لازم يكون عندي Client Program ممكن يكون ... , chrome , Firefox وبيبقى قصاد ال Client Program دا حاجه اسمها Server Program ال بنسماها Apache Web (server) ايه بقي ال بيحصل؟ ال Client Program بيبعت HTTP Request ودا بيحدد انت عايز تشوف ايه من علي السيرفر دا وبعدين السيرفر يرد عليه ب HTTP Response

-- طبعا بيستخدم TCP Protocol لان مينفعش يحصل Loss ودايما ال بيفتح ال Connection هو ال Client مع السيرفر وبعدين بيعت Request ويستني Response By default بيشتغل على 80 = port number

HTTP is "stateless"

- server maintains *no* information about past client requests

-- HTTP مصمم انه Stateless بمعنى انه كل ريكويست ملوش علاقه بال قبله يعني مفيش اي ارتباط بين واحد والتاني

❖ في عندنا طريقتين لل HTTP Connections :

1- Non-Persistent HTTP :

Non-persistent HTTP

- TCP connection opened
 - at most one object sent over TCP connection
 - TCP connection closed
- downloading multiple objects required multiple connections

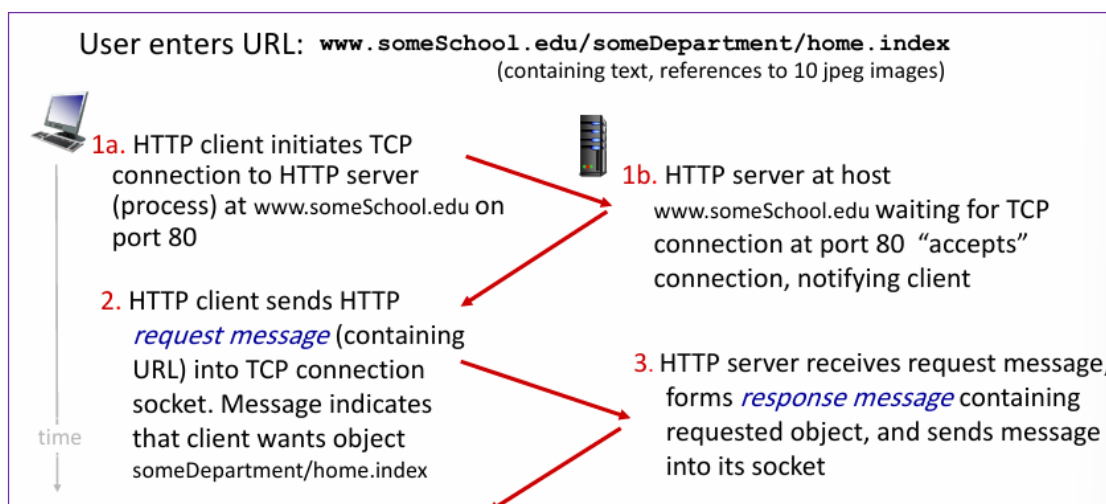
بنبتدي نعمل ايه؟ ان احنا بنفتح Connection ونبعت نجيب ال اوبجكت وبعد ما نجيبه بنقل ال Connection بمعنى ان كل اوبجكت بفتحله connection واستني السيرفر ي accept بعدين اجيب ال اوبجكت بعدين اقل ال connection

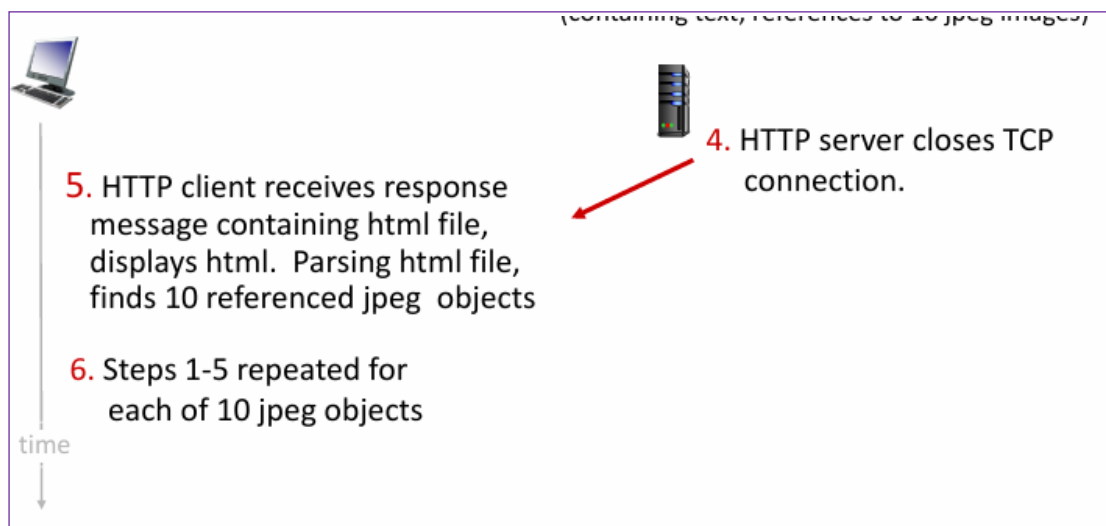
2- Persistent HTTP :

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

هنا بدل مانا كل شوية هفتح connections مانا افتح Single TCP ف الاول انزل كل الاوبجكتس بتوعي ورا بعض وبعدين اقلها



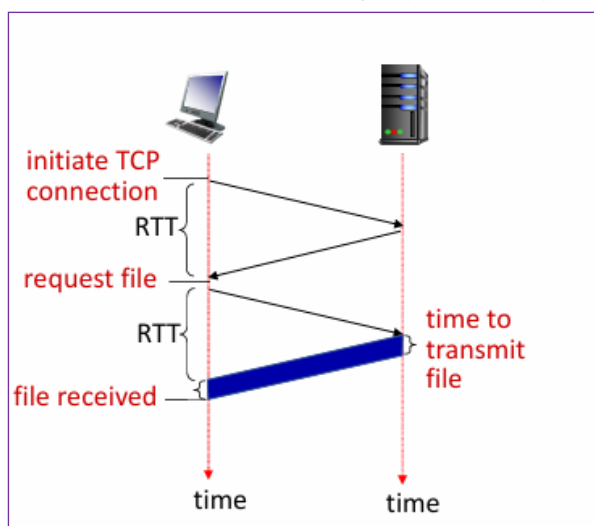


- اول حاجة بتحصل انه هييعت TCP Connection ويستني السيرفر ي accept الكونيكشن دي وبعدين هييعت اجيب ال HTTP ف هييعت اجيب الصفحة وبعد ما ييعتلي الصفحة هييقفل الكونيكشن وبعدين اعيد الكلام دا 10 مرات ورا بعض لكل اوبجيكت

بالتالي لو حسبنا ال Total Time للاوبجيكت هنلاقيه عبارة عن 3 حاجات

بالتالي اقدر احسب ال Response Time

→ RTT (definition): time for a small packet to travel from client to server and back

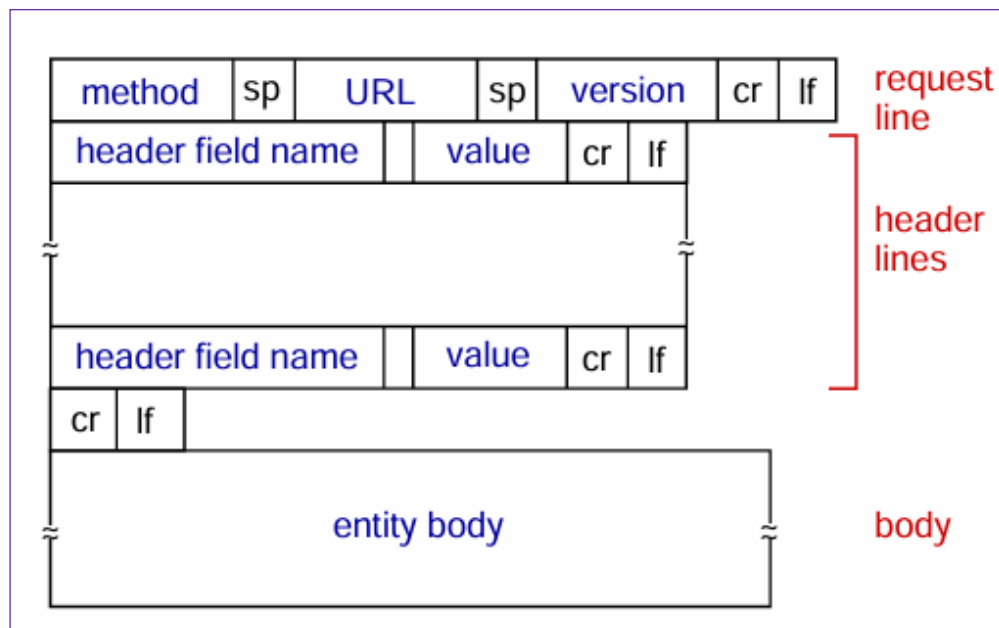


Non-persistent HTTP response time = 2RTT + file transmission time

** لو عندي n-Objects هضرب الرقم دا ف n

--> في حالة ال Persistent بيحاول يوفر زمن ال initiate TCP Connection بدل ما يعمل مع كل اوبجيكت بيعمله ف الاول وبعدين يجيب كل اوبجيكت بعيدين يقفله ف الآخر

HTTP request message: general format



GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

- دي لما تطلب معلومات من السيرفر (زي لما تبث عن حاجة في جوجل)، بتتبع البيانات اللي بتبث عنها في نفس الرابط اللي فوق. زي مثال البحث عن "monkeys & banana" اللي في الصورة، ده بنشوفه في الرابط نفسه.

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

- دي لما تيجي تدخل بيانات في نموذج على موقع (زي لما تكتب اسمك والباسورد)، فالموقع بياخد البيانات دي ويبعثها للسيرفر عشان يخزنها أو يعالجها. البيانات بتتبع في الخلفية ومش بتظهر في الرابط اللي فوق.

HEAD method:

- requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

- دي بتستخدم لما تكون عاوز معلومات عن الصفحة زي "هي الصفحة دي شغالة؟" أو "محتواها إيه؟" بس من غير ما تجيب محتوى الصفحة نفسه يعني زي ما نقول للسيرفر: "ادينني بس الهيدر (المعلومات) بتاعة الصفحة، مش محتاج المحتوى"

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

- دي لما تحب ترفع حاجة جديدة على السيرفر أو تعدل حاجة موجودة زي ما تكون بترفع صورة جديدة على موقع أو بتعدل ملف قديم، والطريقة دي بتستبدل الملف القديم بالكامل بالجديد

➤ HTTP response status codes:

- 200 OK
 - request succeeded, requested object later in this message
- 301 Moved Permanently
 - requested object moved, new location specified later in this message (in Location: field)
- 400 Bad Request
 - request msg not understood by server
- 404 Not Found
 - requested document not found on this server
- 505 HTTP Version Not Supported

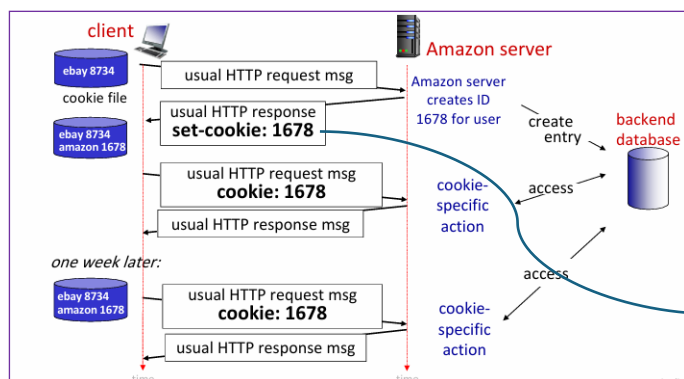
معناها ان الريكويست صح بس السيرفر فيه مشكله

➤ Cookies :

Web sites and client browser use cookies to maintain some state between transactions

■ Four Components:

- 1) cookie header line of HTTP response message
- 2) cookie header line in next HTTP request message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site



- الحكايه والروايه هنفترض اني دخلت علي ويبسايث اول مرة ادخل عليه ف عادي هبتدي ابعت الريكويست ف السايث هيكتشف اني اول مرة ادخل عنده ف اوتوماتيك هيبدي يديني

Id فكل مرة هروح اطلب فيها ويب بيدج ويرد

عليها بيحطلي في ال Response ال Header دا ف البراوزر هيسجلها ف text file عنده باسم السايث ال انا فيه ف لو رجعت بعد فترة ادخل لنفس السايث تاني هيبقي مرتبط سجل الزيارات بتاعتي ب ال Id ال هو اداهوني

❖ Cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

- من عيوبها انها تفقد تعمل Tracking لل web browser بتاعي وتظهر لي اعلانات ليها علاقه بال browser History ال عندي

o Cookies can be used to:

- track user behavior on a given website (first party cookies)
- track user behavior across multiple websites (third party cookies)

without user ever choosing to visit tracker site (!)

- tracking may be invisible to user:

• rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

→ Web Caches :

تخيل معايا كدا أنك بتطلب بيتزا وعندك مخزن صغير في بيتك بتحط فيه قطع بيتزا متبقية.

وفي مطعم ودا المكان اللي بيصنع البيتزا الأصلية (السيرفر). وأنت الشخص اللي عايز ياكل بيتزا.

إيه بقي اللي بيحصل:

لو اول مرة تطلب بيتزا:

بتروح للمخزن الصغير (الويب كاش) بتاعك وتطلب البيتزا ولأن المخزن فاضي، يروح للمطعم (السيرفر) يجيب البيتزا وبعد ما يجيب البيتزا من المطعم بيحط جزء منها في المخزن عشان المرة الجاية. بعدين بيجيبك الجزء اللي طلبته.

المرة الثانية تطلب نفس البيتزا:

بتروح للمخزن تاني تطلب نفس البيتزا بس المرة دي هيلاقها موجودة في المخزن فبيجيبها لك على طول من غير ما يروح للمطعم تاني.

إيه الاستفادة بقي من الهري دا كله؟

- سرعة: بدل ما تستنى وقت طويل عشان المطعم يعمل لك البيتزا من جديد، بتأخذها من المخزن على طول.

توفير في ال Bandwidth: بمعنى إنك مش هتستهلك نفس كمية النت كل مرة تطلب فيها نفس البيتزا.

نطبق بقي المثال دا علي الكمبيوتر عندنا

المخزن الصغير :هو الويب كاش في جهاز الكمبيوتر أو في شبكة.

البيتزا :هي صفحات الويب والصور والفيديوهات.

المطعم :هو السيرفر اللي بيحتوي على هذه الملفات.

- ليه هنستخدم ال Web Cach؟

تحسين سرعة تصفح الإنترنت :بتخلي صفحات الويب تفتح بسرعة أكبر.

تقليل الحمل على السيرفرات :بحيث السيرفرات تعرف تتعامل مع عدد أكبر من المستخدمين.

توفير في استهلاك الإنترنت :خصوصاً لو كنت بتتصفح نفس الصفحات بشكل متكرر.

- باختصار خالص :

الويب كاش زي مخزن صغير بيخزن صفحات الويب اللي بتزورها عشان يقدر يجيبها لك بسرعة في المرات الجاية من غير ما يحتاج يروح للسيرفر كل مرة.



لا تنسو الدعاء لآخواننا في
فلسطين و غزة و العراق و
لبنان و سوريا و السودان.
و تذكر دائما المقاطعة ولا
تتخلي عن مبادئك و انت بتجاهد
معاهم بمقاطعتك.